

개정판
으뜸 파이썬



9장 예외 처리와 파일

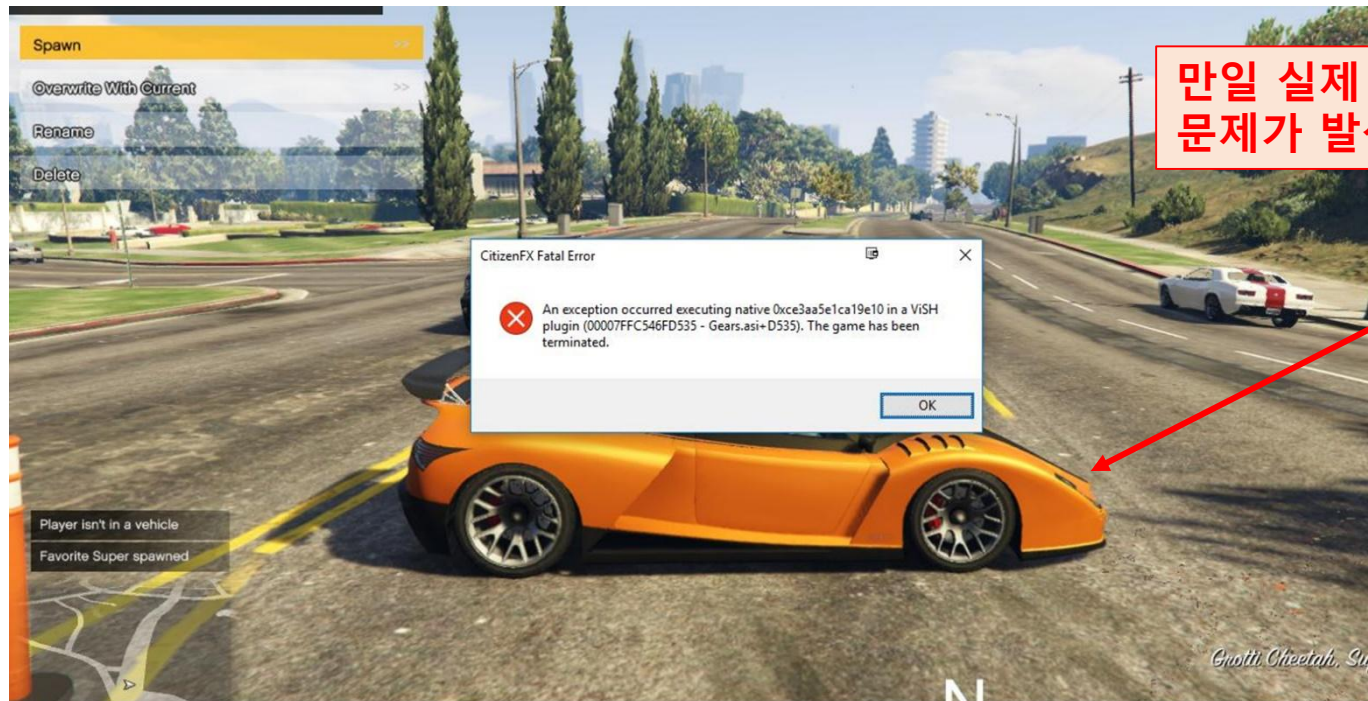
학습목표

- 프로그램 작성시 발생 가능한 구문 오류와 예외에 대해 알아본다.
- 프로그램이 문법적으로 올바르더라도 발생할 수 있는 예외에 대해서 살펴본다.
- 예외를 처리하기 위해서 try-except 문에 대해서 배워보고 직접 활용할 수 있다.
- 파이썬에서 제공하는 예외 이외의 예외를 발생시킬 때에 사용할 수 있는 raise 문에 대해서 이해하고, 활용한다.
- 파일의 개념에 대해 이해한다.
- 파일 입력과 출력을 위한 함수에 대해 알아보고, 이들을 이용하여 파일을 생성하고, 저장하고, 읽어 들일 수 있다.
- 파일의 다룰 때에 사용되는 파일 입출력 모드에 대해 이해한다.
- 파일 입출력 시 발생가능한 예외에 대하여 알아본다.
- 파일 입출력과 관련된 코드를 더욱 효율적이며 가독성 높게 구현할 수 있도록 하는 with 문에 대해 이해하고 사용할 수 있다.

오류와 예외처리의 필요성

- 오류

- 어떤 프로그래밍 언어에서 정해진 문법을 따르지 않는 명령이 입력되어 프로그램이 문제를 일으키는 것



만일 실제 도로를 주행하는 자동차의 컴퓨터에 이런 문제가 발생한다면 생명의 위협을 초래할 수도 있다

오류의 종류와 특징

- 구문 오류 **Syntax Error**

- print() 함수에서 따옴표를 빼먹는 등의 오류
- 문법 오류 라고도 함
- 값 오류, 인덱스 오류, 0으로 나누기 오류, 파일 가져오기 오류 등 단순 오류는 미리 찾아서 고칠 수 있다
- 많은 프로그램에서 오류를 수행 전에 찾기 어려운 경우가 많다.

```
print('Hello world! ) # 닫는 따옴표가 빠져서 구문 오류가 발생하는 코드
```

```
...
```

```
SyntaxError: EOL while scanning string literal
```

자주 볼 수 있는 구문오류의 예

list는 파이썬의 예약어 인데 이것을 리스트 객체를 지칭하는 용도로 사용함

따라서 list() 함수를 호출할 경우 내장함수 list()가 호출되지 않고 위에서 선언한 리스트 객체를 호출함
- 타입에러가 발생하게 됨

```
>>> list = []  
>>> numbers = list(range(10))  
Traceback (most recent call last):  
  File "<pyshell#1>", line 1, in <module>  
    numbers = list(range(10))  
TypeError: 'list' object is not callable
```



NOTE : 구문오류와 예외

위에서 살펴본 바와 같이 문자열에서 닫는 따옴표를 빼먹는다던가, if 문의 비교 후 콜론을 빼먹는 등의 파이썬 오류는 정해진 문법을 지키지 않았기 때문에 발생하므로 구문 오류, 문법 오류 혹은 **파싱 오류**parsing error라고 한다. 그러나 문법적으로는 올바른 프로그램이 수행도중 예상하지 못한 입력 값이 들어오거나 비정상적 상황으로 인해 종료되는 경우도 있을 수 있다. 이러한 경우의 오류를 **예외**exception라고 지칭한다.

예외 발생 예1

- 파이썬 인터프리터 **interpreter**에서 오류 정보 제공
- 프로그램은 오류를 **역추적 Traceback**하여 오류가 발생한 곳에서 오류의 종류를 출력

대화창 실습 : 0으로 나누는 오류

```
>>> a, b = input('두 수를 입력하시오: ').split()
```

두 수를 입력하시오: 10 0

```
>>> result = int(a) / int(b)
```

0으로 나누는 오류

```
Traceback (most recent call last):
```

```
....
```

```
result = int(a) / int(b)
```

0으로 나누는 오류

```
ZeroDivisionError: division by zero
```

이 코드는 문법상 문제가 없음
수행도중 입력값으로 b가 0이 들어왔기 때문에 문제가 발생하였음

예외 발생 예2

- 정수 400, 200대신 '사백', '이백'과 같은 문자열을 입력하여 잘못된 문자열 형식의 자료를 숫자로 바꾸는 경우 발생하는 오류

대화창 실습 : 문자열과 숫자를 덧셈하는 오류

```
>>> a, b = input('두 수를 입력하시오: ').split()
```

두 수를 입력하시오: 사백 이백

```
>>> result = int(a) / int(b)
```

Traceback (most recent call last):

....

```
result = int(a) / int(b)
```

```
ValueError: invalid literal for int() with base 10: '사백'
```

이 코드는 문법상 문제가 없음
수행도중 입력값으로 a, b에 잘못된 값이 들어왔기 때문에 문제가 발생하였음

try-except 문의 문법

- 예외처리 **exception handling**를 위해 사용
- try 절 - 예외가 발생할 우려가 있는 코드를 입력
- except 절 - 오류가 발생했을 때 처리할 내용을 담음
 - 예외가 발생하지 않으면 except를 건너뛰고 예외가 발생하면 오류를 확인하여 except의 매칭되는 부분으로 넘겨줌

```
try:  
    {예외가 발생할 우려가 있는 코드}  
except [예외의 타입]:  
    {예외가 발생할 경우 실행되는 코드}
```

try-except 구문을 이용한 예외처리 예

- 프로그램은 급작스럽게 종료되지 않고 사용자에게 대처할 수 있는 방법을 제시할 수 있으며, 이전의 프로그램에 비해 안정적으로 작동함

코드 9-1 : try-except 구문을 이용한 예외처리

div_with_try_except.py

try:

```
a, b = input('두 수를 입력하시오: ').split()
result = int(a) / int(b)
print('{} / {} = {}'.format(a, b, result))
```

except:

```
print('수가 정확한지 확인하세요.')
```

실행결과

두 수를 입력하시오: 10 0
수가 정확한지 확인하세요.

실행결과

두 수를 입력하시오: 사백 이백
수가 정확한지 확인하세요.

실행결과

두 수를 입력하시오: 50 2
50/2 = 25.0

이전 코드보다 훨씬
안전하게 동작함

파이썬의 예외 목록

BaseException
 Exception
 ArithmeticError
 FloatingPointError
 OverflowError
 ZeroDivisionError
 AssertionError
 AttributeError
 BufferError
 EOFError
 ImportError
 ModuleNotFoundError
 LookupError
 IndexError

ProcessLookupError
 TimeoutError
 ReferenceError
 RuntimeError
 NotImplementedError
 RecursionError
 StopIteration
 StopAsyncIteration
 SyntaxError
 IndentationError
 TabError
 SystemError
 TypeError
 ValueError

파이썬의 예외 목록

KeyError
 MemoryError
 NameError
 UnboundLocalError
 OSError
 BlockingIOError
 ChildProcessError
 ConnectionError
 BrokenPipeError
 ConnectionAbortedError
 ConnectionRefusedError
 ConnectionResetError
 FileExistsError
 FileNotFoundError
 InterruptedError
 IsADirectoryError
 NotADirectoryError
 PermissionError

UnicodeError
 UnicodeDecodeError
 UnicodeEncodeError
 UnicodeTranslateError
 Warning
 BytesWarning
 DeprecationWarning
 FutureWarning
 ImportWarning
 PendingDeprecationWarning
 ResourceWarning
 RuntimeWarning
 SyntaxWarning
 UnicodeWarning
 UserWarning
 GeneratorExit
 KeyboardInterrupt
 SystemExit

예외의 종류 출력하기1

- Exception as e
 - 어떤 예외 상황에 의해서 except가 실행되었는지 알고 싶을 경우 Exception 변수 e를 선언한 후 e 값을 출력하면 오류의 종류를 알 수 있게 됨

코드 9-2 : try-except 문을 사용한 예외처리와 예외의 종류를 출력하기

find_error1.py

try:

b = 2 / 0

a = 1 + 'hundred'

except Exception as e:

print('error :', e)

실행결과

error : division by zero

예외의 종류를 출력하기2

- 지원되지 않은 연산자 +를 정수와 실수 자료형에 적용함으로써 오류가 발생

코드 9-3 : try-except 문을 사용한 예외처리와 예외의 종류를 출력하기

find_error2.py

try:

a = 1 + 'hundred'

except Exception as e:

print('error :', e)

실행결과

error : unsupported operand type(s) for +: 'int' and 'str'



LAB 9-1 : 다음 코드는 어떤 예외를 발생시키는가?

1. 다음 코드는 각각 어떤 예외를 발생시키는가?

```
>>> a = [10, 20, 30]
>>> a[3]
```

```
>>> n = int('20%')
```

```
>>> a = 100 + '200'
```

```
try:
    a=[10,20,30]
    print(a[3])
except Exception as e:
    print('error :', e)
```

```
try:
    n=int('20%')
except Exception as e:
    print('error :', e)
```

```
try:
    a=100+'200'
except Exception as e:
    print('error :', e)
```

2. 위의 문제 1번 코드를 try - except 문으로 감싸고 Exception e를 출력하여 어떤 출력문이 나오는지 확인하고 적으시오.

```
error : list index out of range
error : invalid literal for int() with base 10: '20%'
error : unsupported operand type(s) for +: 'int' and 'str'
```

구체적인 예외를 명시하기1

코드 9-4 : try-except 문을 사용한 구체적인 예외처리

find_error3.py

```
try:
    b = 2 / 0
    a = 1 + 'hundred'
except ZeroDivisionError :
    print('0으로 나누는 오류')
except TypeError :
    print('지원되지 않은 연산자를 사용하는 오류')
```

실행결과

0으로 나누는 오류

구체적인 예외를 명시하기2

- `b = 2 / 0`을 주석처리 할 경우 `TypeError`라는 예외가 발생

코드 9-5 : try-except 문을 사용한 구체적인 예외처리

find_error4.py

```
try:
```

```
#    b = 2 / 0
```

```
    a = 1 + 'hundred'
```

```
except ZeroDivisionError:
```

```
    print('0으로 나누는 오류')
```

```
except TypeError:
```

```
    print('지원되지 않은 연산자를 사용하는 오류')
```

실행결과

지원되지 않은 연산자를 사용하는 오류

구체적인 예외처리와 보편적인 예외처리 문을 가진 문장

- `except` 구체적인 예외 : 와 `except Exception as e`: 보편적인 예외처리 기능을 넣어 주는 것이 더 안전한 코딩이 된다

코드 9-6 : 구체적인 예외처리와 보편적인 예외처리 문을 가진 문장

find_error5.py

try:

`a = 1 + 'hundred'`

except `ZeroDivisionError`:

`print('0으로 나누는 오류')`

except `TypeError`:

`print('지원되지 않은 연산자를 사용하는 오류')`

except `Exception as e`: # 보편적인 예외처리 기능

`print('error :', e)`

`print('나눗셈과 연산자를 다시 확인하세요')`

실행결과

지원되지 않은 연산자를 사용하는 오류

try-except-else 문

코드 9-7 : 복수의 except 문과 else 문을 포함한 예외처리

try_except_else_test.py

try:

```
a, b = input('두 수를 입력하시오: ').split()
```

```
result = int(a) / int(b)
```

except ZeroDivisionError:

```
print('오류 : 0으로 나눴습니다.')
```

except ValueError:

```
print('오류 : 입력 값이 정수나 실수가 아닙니다.')
```

except:

```
print('오류 : 10 2와 같이 두 정수를 입력하세요.')
```

else:

```
print('{} / {} = {}'.format(a, b, result))
```

- 예외가 발생하지 않을 경우 실행할 문장을 else에 기록

실행결과

두 수를 입력하시오: 20 0

오류 : 0으로 나눴습니다.

실행결과

두 수를 입력하시오: 10 2

10 / 2 = 5.0



LAB 9-2 : 다음 코드는 어떤 예외를 발생시키는가?

1. 다음 코드가 있을 경우 이 코드에 적합한 예외 처리문을 만드시오

```
10 * ( 30 / 0 )
```

```
x = int(input('정수 x를 입력하세요: '))
```

```
import sys
```

```
f = open('myfile.txt')
```

```
s = f.readline()
```

LAB 9-2 : , 3번 정답 코드

```
import sys
try:
    f = open('myfile.txt')
    s = f.readline()
except:
    print("파일이 없습니다")
```

LAB 9-2 : , 1번 정답 코드

```
try:
    Test = 10 * (30 / 0)
except ZeroDivisionError:
    print("0으로 나눴습니다.")
```

LAB 9-2 : 2번 정답 코드

```
try:
    x = int(input("정수 x를 입력하세요: "))
except:
    print('오류: 정수를 입력하세요')
```

try-except-finally 문

- except와 else의 차이는 예외의 발생 여부
- finally는 예외의 발생 여부와 상관없이 항상 실행

이름	설명
try:	먼저 실행되어 예외가 발생하지 않으면 except를 건너뛰고, 예외가 발생하면 오류를 확인하여 except의 매칭되는 부분으로 넘겨준다.
except:	예외가 발생했을 때 처리할 내용을 담는다.
else:	예외가 발생하지 않을 때 실행하게 되는 블록이다.
finally:	예외의 발생 여부와 상관없이 항상 실행되는 블록이다.

try-except-else-finally 구문의 사용법

코드 9-8 : try-except-else-finally 구문의 사용법

divide_func.py

```
def divide(x, y):  
    try:  
        result = x / y  
    except ZeroDivisionError:  
        print('0으로 나누는 오류발생')  
    else:  
        print('결과 :', result)  
    finally:  
        print('수행완료')
```

```
print('divide(100, 2) 함수호출 :')  
divide(100, 2)  
print('divide(100, 0) 함수호출 :')  
divide(100, 0)
```

실행결과

divide(100,2) 함수호출 :

결과 : 50.0

수행완료

divide(100,0) 함수호출 :

0으로 나누는 오류발생

수행완료

예외 발생 여부와 상관없이 “항상” 실행됨



LAB 9-3 : 예외의 생성과 처리

1. [1, 2, 3, 4, 5]의 다섯개의 요소를 가진 리스트 a가 있을 때 다음과 같이 사용자로 부터 정수를 입력 받은 후 이에 해당하는 인덱스를 출력하도록 하자.

```
a = [1, 2, 3, 4, 5]
```

a의 요소를 하나 선택하시오 : 2

2 은(는) 두 번째 요소입니다.

```
a = [1, 2, 3, 4, 5]
```

a의 요소를 하나 선택하시오 : 1

1 은(는) 첫 번째 요소입니다.

LAB 9-3 : 1번 코드

```
a = [1, 2, 3, 4, 5]
```

```
print("a =", a)
```

```
try:
```

```
    idx = int(input("a의 원소를 하나 선택하시오 : "))
```

```
    print("{} 은(는) {} 번째 원소입니다.".format(idx, a.index(idx)+1))
```

```
except:
```

```
    print("a의 원소에 {}(이)가 없습니다.".format(idx))
```

2. 다음과 같이 '삼'이 입력되면 try - except 문을 통해 '오류 : 입력 값이 정수나 실수가 아님'을 출력하시오.

```
a = [1, 2, 3, 4, 5]
```

a의 요소를 하나 선택하시오 : 삼

오류 : 입력 값이 정수나 실수가 아님

LAB 9-3 : 2번 코드

```
a = [1, 2, 3, 4, 5]
```

```
print("a =", a)
```

```
try:
```

```
    idx = int(input("a의 원소를 하나 선택하시오 : "))
```

```
    print("{} 은(는) {} 번째 원소입니다.".format(idx, a.index(idx)+1))
```

```
except:
```

```
    print("오류:입력 값이 정수나 실수가 아님")
```

내장 예외 Built-in Exception

- Exception은 클래스로 정의되어 있고, 계층적 구조로 만들어져 있기 때문에 새로운 예외를 만들 때에는 Exception을 상속받아서 구현함

```
BaseException
+-- SystemExit
+-- KeyboardInterrupt
+-- GeneratorExit
+-- Exception
    +-- StopIteration
    +-- StopAsyncIteration
    +-- ArithmeticError
    |   +-- FloatingPointError
    |   +-- OverflowError
    |   +-- ZeroDivisionError
    +-- AssertionError
    +-- AttributeError
    +-- BufferError
    +-- EOFError
    +-- ImportError
    |   +-- ModuleNotFoundError
    +-- LookupError
    |   +-- IndexError
    |   +-- KeyError
    +-- MemoryError
    +-- NameError
    |   +-- UnboundLocalError
    +-- OSError
    |   +-- BlockingIOError
    |   +-- ChildProcessError
    |   +-- ConnectionError
    |   |   +-- BrokenPipeError
    |   |   +-- ConnectionAbortedError
    |   |   +-- ConnectionRefusedError
    |   |   +-- ConnectionResetError
    |   +-- FileExistsError
    |   +-- FileNotFoundError
    |   +-- InterruptedError
    |   +-- IsADirectoryError
    |   +-- NotADirectoryError
    |   +-- PermissionError
    |   +-- ProcessLookupError
    |   +-- TimeoutError
    +-- ReferenceError
    +-- RuntimeError
    |   +-- NotImplementedError
    |   +-- RecursionError
    +-- SyntaxError
    |   +-- IndentationError
    |   +-- TabError
```

raise 문

- raise를 이용하여 예외를 강제로 발생시킴
- raise 문에 의해서 Exception이 발생되어 파이썬 인터프리터 프로그램이 그 과정을 추적Traceback하였다는 출력이 뜸

대화창 실습 : raise 문을 이용하여 예외 발생시키기1

```
>>> raise Exception('Exception raised') #예외 문구는 한글도 가능
```

```
Traceback (most recent call last):
```

```
  File "<stdin>", line 1, in <module>
```

```
    raise Exception('Exception raised')
```

```
Exception: Exception raised
```


raise를 이용한 예외의 발생

코드 9-9 : raise를 이용한 예외의 발생

raise_test.py

```
def get_ans(ans):  
    if ans in ['예', '아니오']:  
        print('정상적인 입력입니다.')  
    else:  
        raise ValueError('입력을 확인하세요.')  
while True:  
    try:  
        ans = input('예/아니오 중 하나를 입력하세요 :')  
        get_ans(ans)  
        break  
    except Exception as e:  
        print('error :', e)
```

실행결과

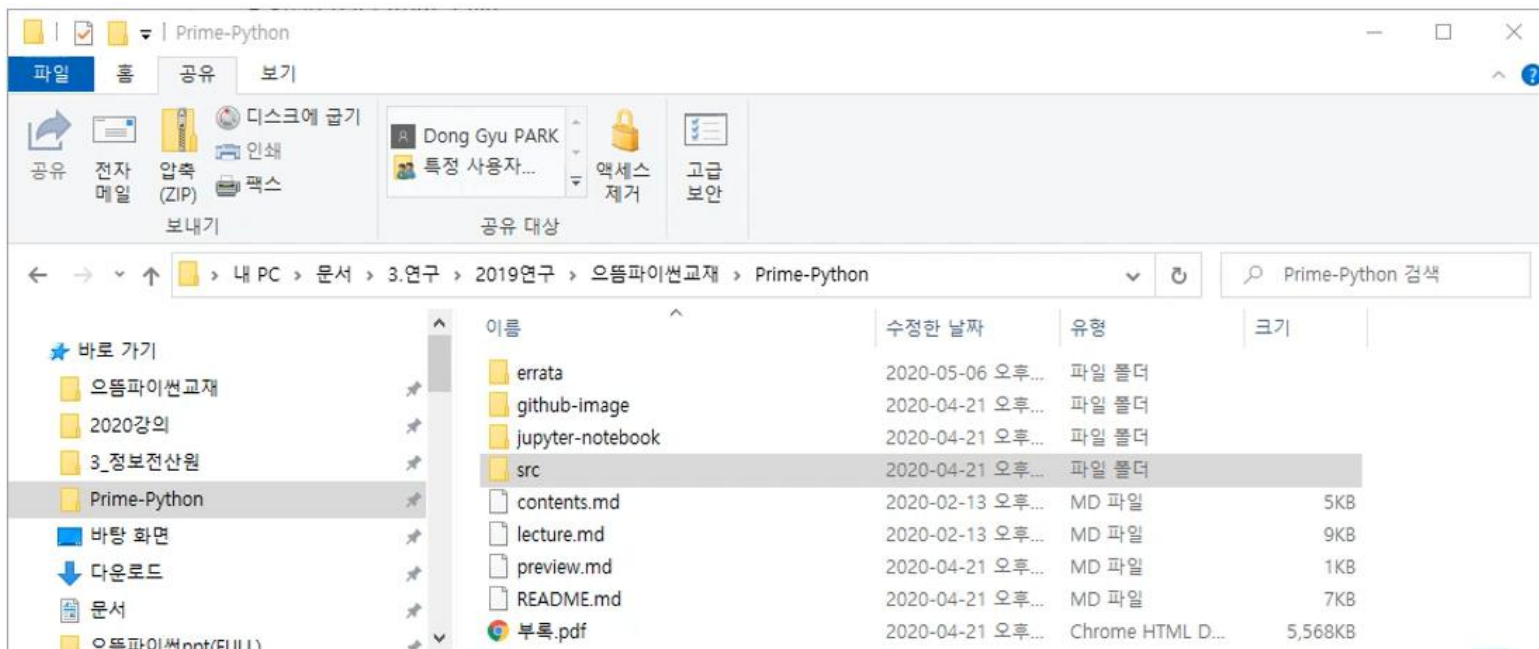
```
예/아니오 중 하나를 입력하세요 :예-쓰  
error : 입력을 확인하세요.  
예/아니오 중 하나를 입력하세요 :노우  
error : 입력을 확인하세요.  
예/아니오 중 하나를 입력하세요 :예  
정상적인 입력입니다.
```

‘예’나 ‘아니오’ 이외의 ans 값이 있을 경우 ValueError()라는 예외 객체를 발생시킨다

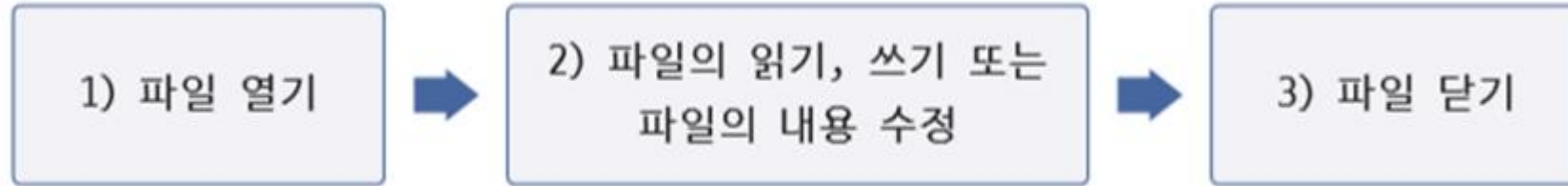
예외가 발생하면 이 코드가 처리함

파일 file이란 무엇인가

- 컴퓨터의 저장 장치 내에 데이터를 저장하기 위해 사용하는 논리적인 단위
- 영구 보존을 위한 저장 장치(하드디스크 hard disk나 외장 디스크 external disk)에 파일이란 단위를 사용하여 저장한 후 필요할 때 불러서 사용하는 것이 가능



파이썬에서 파일을 사용하기 위한 절차



1. 파일 **열기** - 저장 장치 내의 경로와 파일 이름을 지정해서 파일을 가져온다

경로path : 컴퓨터 저장 장치 내의 파일 위치

2. 사용 목적에 따라 **사용**

내용 확인 및 새로운 내용 추가 또는 기존 내용 삭제 가능

3. 파일 **닫기** - 파일을 모두 사용하고 나면 메모리를 시스템에서 사용할 수 있도록 반환

파일 열고 닫기

- 형식

파일객체 = `open`(파일명, 파일모드)

.....

파일객체.`close`()

코드 10-10 : 파일 열기와 쓰기, 파일 닫기의 표현

file_write_hello.py

파일명

파일모드

`f = open('hello.txt', 'w')`

#1) 파일을 쓰기 전용으로 열기

`f.write('Hello world!')`

#2) hello.txt 파일에 문자열을 기록

`f.close()`

#3) 파일을 닫기

open() 함수

- 파이썬에서 파일을 열어서 연결하는 역할
- 파일 이름과 파일 오픈 모드 등의 인자를 가질 수 있음
- `encoding`이라는 인자를 통해 파일의 인코딩 형식 지정 가능
- 지정된 경로가 없을 경우 디폴트 경로는 파이썬 스크립트 파일이 있는 곳임

파일 오픈 모드

모드	의미
r	read의 약자로 파일을 읽기 전용 모드로 연다(파일 열기의 기본 모드) - 파일이 없을 경우 에러가 발생 - 읽기 전용 모드이므로 파일에 기록하기를 시도하면 오류가 발생 - 파일의 처음부터 읽음
w	write의 약자로 파일을 쓰기 전용 모드로 연다 - 파일이 없을 경우 파일을 생성하며, 파일이 있을 경우 덮어쓰게 된다(주의) - 파일의 처음 위치에 기록
a	append의 약자로 파일을 쓰기 모드 로 열어서 파일의 끝에 내용을 추가 - 기존 파일이 없을 경우 새로 파일을 만들어서 추가
x	쓰기 전용 으로 파일을 여는데, 파일을 새로 만들기 위해서 사용한다 - 파일이 있을 경우 에러가 발생한다(w 모드의 안전모드)

파일 오픈 모드

파일 모드	의미
+	+ 기호는 읽기 쓰기 모드('r+'나 'w+', 'r+t'와 같은 조합으로 사용 가능) 'r+' : 읽기 모드로 열어서 쓰기까지 가능 'w+' : 쓰기 모드로 열어서 읽기까지 가능 읽기와 쓰기로 모드를 변경하려면 seek()를 사용해야 함
t	text의 약자로 텍스트 파일형식으로 파일을 열거나 생성한다 파일 열기의 기본 모드이다
b	binary의 약자로 이진 파일형식으로 파일을 열거나 생성

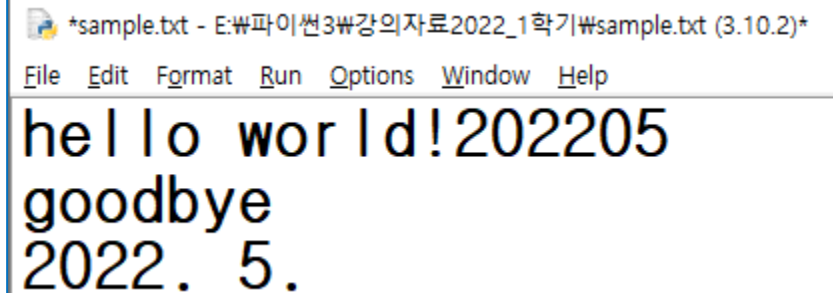
파일에 기록하기

- write() 메소드
 - write('파일에 기록할 내용')
 - 자동 개행(줄바꿈)을 하지 않음
- print() 내장함수
 - print('파일에 기록할 내용', file=파일객체)
 - file 옵션을 사용하여 파일을 지정해야 함

```
f=open('sample.txt','w')
f.write('hello world!')
print('202205',file=f)
f.write('goodbye\n')
f.write('2022. 5.')

f.close()
```

실행 결과



```
*sample.txt - E:\₩파이썬3₩강의자료2022_1학기₩sample.txt (3.10.2)*
File Edit Format Run Options Window Help
hello world!202205
goodbye
2022. 5.
```


파일에 한글을 기록

- open() 함수에 encoding='utf-8'을 기록

```
f=open('sample.txt','w',encoding='UTF8')  
f.write('hello world!')  
print('202205',file=f)  
f.write('goodbye\n')  
f.write('2022년 5월')  
  
f.close()
```

실행 결과

sample.txt - E:\파이썬3\강의자료\2022_1학기\sample.txt (3.10.2)

File Edit Format Run Options Window Help

hello world!202205

goodbye

2022년 5월



NOTE : 버퍼의 필요성과 close() 함수

컴퓨터의 하드디스크를 통해 데이터를 읽고 쓰는 동작은 중앙처리장치나 메모리 내에서 데이터를 읽고 쓰는 것보다 수십 배에서 수백 배 이상 느리다. 따라서 가급적 데이터를 디스크에 읽고 쓰는 일을 한꺼번에 모아서 처리하는 것이 더 빠르다. 이 때문에 컴퓨터의 메모리에 있는 1000 바이트 데이터를 1 바이트씩 1000번 디스크에 저장하는 것 보다 100 바이트 단위로 10번 저장하는 것이 더 빠르다.

따라서 버퍼를 사용하여 write 명령으로 들어온 데이터를 100 바이트 단위로 모아 두었다가 100 바이트가 되면 한 번에 쓰는 것이 일반적으로 더 효과적이다. close() 함수가 하는 일은 이제 이 파일에 더 이상 write 명령을 보낼 일이 없다는 정보를 줌으로서 버퍼에 있는 정보를 디스크에 저장하는 역할을 한다.



LAB 9-4 : 파일 저장하기

1. 다음과 같은 내용을 가지는 numbers.txt 파일을 파이썬의 write() 메소드를 이용하여 작성하여라. 이때 아래와 같이 줄바꿈이 되도록 하자.

```
100
200
300
400
```

2. 다음과 같은 내용을 가지는 we_will_rock.txt 파일을 파이썬의 write() 메소드를 이용하여 작성하여라. 따옴표의 출력에 주의하여 프로그램을 작성하자.

```
Buddy, you're a boy, make a big noise
Playing in the street, gonna be a big man someday
You got mud on your face, you big disgrace
Kicking your can all over the place, singin'
We will, we will rock you
We will, we will rock you
```

LAB 9-4 , 1번 코드

```
f = open('numbers.txt', 'w')
f.write('100Wn')
f.write('200Wn')
f.write('300Wn')
f.write('400')
f.close()
```

LAB 9-4 , 2번 코드

```
f = open('we_will_rock.txt', 'w')
f.write("Buddy, you're a boy, ")
f.write("make a big noisePlaying in the street,")
f.write("gonna be a big manWn")
f.write('face, you big disgraceWn')
f.write('Kicking your can all over the place, singinW'Wn')
f.write("We will, we will rock youWe will, we will rock youWn")
f.close()
```

파일에 기록된 내용 읽기

- 인수 없는 read() 메소드
 - 파일의 모든 문자열(전체 내용)은 줄 바꿈에 관계없이 하나의 문자열로 반환됨

코드 9-11 : 파일 열기와 읽기, 파일 닫기의 표현

file_read_hello.py

```
f = open('st.txt', 'r')  
  
s = f.read()    # hello.txt 파일을 읽는다.  
  
print(s)  
  
f.close()
```

실행결과

hello world!

파일에 기록된 내용 읽기

- 인수(수)를 사용한 read() 메소드
 - 인수에 해당하는 개수 만큼의 문자열을 읽어옴

코드 9-12 : read() 메소드를 이용하여 지정된 문자 크기만큼 읽기

file_read_5char.py

```
f = open('hello.txt', 'r')  
  
s = f.read(5)      # hello.txt 파일을 읽는다.  
print(s)  
  
s = f.read(5)      # hello.txt 파일을 읽는다.  
print(s)  
  
f.close()
```

실행결과

Hello
worl

파일에 기록된 내용 읽기

- `readlines()` 메소드 사용
 - 파일의 내용을 줄 단위로 읽어서 리스트 형태로 저장

```
f = open('foo.txt', 'r')
s = f.readlines()
    # 파일의 내용을 줄 별로 읽어서 리스트 형태로 저장
print( s )
f.close()
```

실행결과

```
['AAWn', 'BBWn', 'CCC']
```

foo.txt 파일의 내

용

AAA

BBB

CCC

파일에 기록된 내용 읽기

- readline() 메소드

- 한줄씩 읽어옴

코드 9-13 : readline() 메소드를 이용한 줄 단위 읽기와 출력하기

file_readline.py

```
f = open('foo.txt', 'r')
s = f.readline()           # 파일의 첫 번째 줄을 읽어온다
print( s, end='' )        # 첫 번째 줄의 내용을 화면에 출력
s = f.readline()           # 파일의 두 번째 줄을 읽어온다
print( s )                 # 두 번째 줄의 내용을 화면에 출력
f.close()
```

foo.txt 파일의 내용

AAA
BBB
CCC

실행결과

AAA

BBB

빈 줄을
없애기 위해
end="" 사용

파일 처리 예 (매출 처리)

```
1 infilename = input("자료 파일 이름: ")
2 outfilename = input("결과 저장 파일 이름: ")
3
4 # 입력과 출력 파일을 연다
5 infile = open(infilename, "r")
6 outfile = open(outfilename, "w")
7
8 tot = 0 ; count = 0
9
10 # 입력 파일에서 한 줄을 읽어서 합계를 계산
11 line = infile.readline()
12 while line != "":
13     s = int(line)
14     tot += s ; count += 1
15     line = infile.readline()
16
17 # 총매출과 일평균 매출을 출력 파일에 기록한다.
18 outfile.write("총매출 = " + str(tot) + "\n")
19 outfile.write("평균 일매출 = " + str(tot/count) + "\n")
20
21 infile.close() ; outfile.close()
```

sales.txt

파일	편집	보기
1000000		
1000000		
1000000		
500000		
1500000		

상점의 하루 매출
한줄에 정수로 기록

IDLE Shell 3.12.1

File Edit Shell Debug Options Window Help

Python 3.12.1 (tags/v3.12.1:2305ca1 AMD64) on win32
Type "help", "copyright", "credits"

>>> = RESTART: G:/파이썬3/강의자료2024_
자료 파일 이름: sales.txt
결과 저장 파일 이름: result.txt

>>>

result

파일	편집	보기
총매출 = 5000000		
평균 일매출 = 1000000.0		

상점의
총매출과
평균 일매출

rstrip() 메소드

- 스트링의 마지막에 나오는 문자 삭제

코드 9-14 : readline() 메소드와 rstrip()을 이용한 줄 단위 읽기와 출력

file_readline_rstrip.py

```
f = open('foo.txt', 'r')
```

```
s = f.readline().rstrip() # 'AAA' 줄을 읽고 오른쪽 줄 바꿈 문자를 지움
```

```
print(s)
```

```
s = f.readline().rstrip() # 'BBB' 줄을 읽고 오른쪽 줄 바꿈 문자를 지움
```

```
print(s)
```

```
f.close()
```

실행결과

AAA

BBB



LAB 9-5 : 파일 읽어오기

1. 다음과 같은 내용을 가지는 numbers.txt 파일을 파이썬의 read() 메소드를 이용하여 출력하시오(힌트 : f.readline().rstrip()을 사용하라).

```
100
200
300
400
```

2. 다음과 같은 내용을 가지는 we_will_rock.txt 파일을 파이썬의 read() 메소드를 이용하여 출력하라.(힌트 : f.readline().rstrip()을 사용하라)

```
Buddy, you're a boy, make a big noise
Playing in the street, gonna be a big man someday
You got mud on your face, you big disgrace
Kicking your can all over the place, singin'
We will, we will rock you
We will, we will rock you
```

LAB 9-5 , 1번 코드

```
f = open('numbers.txt', 'r')
for _ in range(4):
    s = f.readline().rstrip()
    print(s)
```

LAB 9-5 , 2번 코드

```
f = open('we_will_rock.txt', 'r')
for _ in range(6):
    s = f.readline().rstrip()
    print(s)
f.close()

'''
f = open('we_will_rock.txt', 'r')

s = f.read()
print(s)

f.close()
'''
```

파일에 내용 추가하기

- 'a+' 모드를 사용
 - 이미 만들어진 파일의 뒤쪽에 새로운 내용을 추가
 - a는 추가(append)모드
 - +는 읽기와 쓰기 모두 가능

코드 9-15 : 파일 추가(append) 모드를 이용한 파일 열기와 추가하기

file_append.py

```
f = open('foo.txt', 'a+') # 파일을 추가모드로 열기
f.write('This will be appended.\n')
f.write('This too.\n')
f.close()
```

foo.txt 파일의 내용

AAA
BBB
CCC

추가 전

foo.txt 파일의 내용

AAA
BBB
CCC
This will be appended.
This too.

추가 후

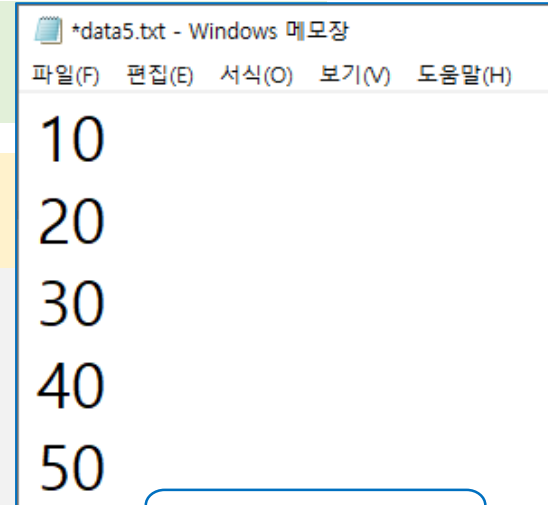
파일 쓰기 예

- 사용자로부터 다섯 개의 정수를 입력으로 받아서 data5.txt에 저장

코드 9-16 : 사용자로부터 입력 받은 다섯 개의 정수를 저장하는 프로그램

file_write_5num.py

```
f = open('data5.txt', 'w')
for _ in range(5):
    n = input('정수를 입력하세요: ')
    f.write(n)    #입력된 값을 data5.txt에 쓰기(문자열)
    f.write('\n')  # 값을 쓴 후 줄 바꿈하기
f.close()        # 파일을 닫는다.
```



실행결과

정수를 입력하세요: 10
정수를 입력하세요: 20
정수를 입력하세요: 30
정수를 입력하세요: 40
정수를 입력하세요: 50

파일 읽기 예

- 파일로부터 정수를 읽어서 정수들의 합과 평균을 출력

코드 9-17 : 파일로 부터 정수를 입력 받아 합과 평균을 계산

file_read_5num.py

```
f = open('data5.txt','r') # 읽기 모드로 파일 열기
su = 0
for _ in range(5):
    n = int(f.readline()) # data5.txt 파일의 한 줄 내의 숫자를 읽어서
    su += n                # su에 누적해서 더한다

print('다섯 숫자의 합 = {}, 평균 = {}'.format(su, su/5))
f.close()                  # 파일을 닫는다.
```

실행결과

다섯 숫자의 합 = 150, 평균 = 30.0

file_numbering1.py

```
fname = input('입력할 파일의 이름 : ')
f = open(fname, 'r')    # 파일 열기
n = 1                   # 줄 번호를 출력하기 위한 변수
l = f.readline()
while l :               # 읽어들이 줄이 있으면 반복 수행함
    print('{:3d}: {}'.format(n, l), end = '') # 줄 번호와 내용을 출력
    n += 1              # 줄 번호를 증가시킴
    l = f.readline()    # 다음 줄을 읽어온다

f.close()
```

실행결과

입력할 파일의 이름 : we_will_rock.txt

1: Buddy, you're a boy, make a big noise

2: Playing in the street, gonna be a big man someday

3: You got mud on your face, you big disgrace

4: Kicking your can all over the place, singin'

5: we will, we will rock you

6: we will, we will rock you

파일의 예외처리

코드 9-19 : try-except 문을 사용한 파일 입출력

file_numbering3.py

```
import sys
try :
    fname = input('입력할 파일의 이름 : ')
    try:
        f = open(fname, 'r', encoding='UTF8')          # 파일 열기
    except IOError:
        print('Could not read file:', fname)
        raise SystemExit('due to IOError')
    except:
        print('Unexpected error:', sys.exc_info()[0])
        raise SystemExit('due to Unexpected error')

    n = 1          # 라인번호 n을 1로 초기화 함
    l = f.readline() # 변수 l은 읽어 들인 한 줄의 문자열을 저장
    while l :
        print('{:3d}: {}'.format(n, l), end='')
        n += 1          # 한 줄을 출력한 후 줄 수를 증가시킨다
        l = f.readline() # 다음 줄을 읽어온다
    f.close()

except SystemExit as e:
    print('Exit', e.args)
```

- r 모드로 파일을 열 경우
파일이 존재하지 않으면
오류 발생
 - 파일명이 제대로 입력되었는가?
 - 파일을 열 수 있는가?

실행결과

```
입력할 파일의 이름 : file_read_5num
Could not read file: file_read_5num
Exit ('due to IOEeeor',)
```



LAB 9-6 : 파일 저장하기

1. 사용자로부터 파일 이름을 입력으로 받은 후 입력받은 파일의 모든 문자를 대문자로 변환한 후 출력하여라. 그리고 이 내용을 [파일명_upper].txt 라는 이름의 파일로 저장하여라(힌트 : 문자열의 upper() 메소드를 사용시오).

입력할 파일의 이름 : we_will_rock.txt

BUDDY, YOU'RE A BOY, MAKE A BIG NOISE

PLAYING IN THE STREET, GONNA BE A BIG MAN SOMEDAY

YOU GOT MUD ON YOUR FACE, YOU BIG DISGRACE

KICKING YOUR CAN ALL OVER THE PLACE, SINGIN'

WE WILL, WE WILL ROCK YOU

WE WILL, WE WILL ROCK YOU

2. 사용자로부터 파일 이름을 입력으로 받은 후 입력받은 파일의 모든 줄을 역순으로 하여 다음과 같이 출력하시오.

파일 이름을 입력하세요 : we_will_rock.txt

We will, we will rock you

We will, we will rock you

Kicking your can all over the place, singin'

You got mud on your face, you big disgrace

Playing in the street, gonna be a big man someday

Buddy, you're a boy, make a big noise

LAB 9-6, 1번 코드

```
fname = input('입력할 파일의 이름 : ')
f = open(fname, 'r') # 파일 열기
n = 1                # 줄 번호를 출력하기 위한 변수
l = f.readline()
while l :             # 읽어들이 줄이 있으면 반복 수행함
    l = l.upper()
    print('{}' .format(l), end='')
    l = f.readline()

f.close()
```

LAB 9-6, 2번 코드

```
fname = input('입력할 파일의 이름 : ')
for line in reversed(list(open(fname))):
    print(line.rstrip())
```


with 문법

- 문법을 명확하게 만들고 예외를 손쉽게 처리하는 방법

코드 9-20 : 파일 열기와 쓰기, 파일 닫기의 표현 1

file_write_hello1.py

```
f = open('hello.txt', 'w') # 파일 열기  
f.write('Hello World!') # hello.txt 파일에 쓰기  
f.close() # 파일을 닫는다.
```

- with open() as 파일객체 : 문법 사용

```
with open('hello.txt', mode='w') as f:  
    f.write('Hello world!')
```

- close() 호출 없이 자동으로 파일 닫아 줌

덮어쓰기 방지

- f.write() 작업은 항상 오류의 가능성을 포함
 - 덮어쓰기를 원하지 않을 경우 x모드를 사용

```
with open('hello.txt', mode='w') as f:  
    f.write('Hello world!')  
  
with open('hello.txt', 'x') as f:  
    f.write('Hi~~~')
```

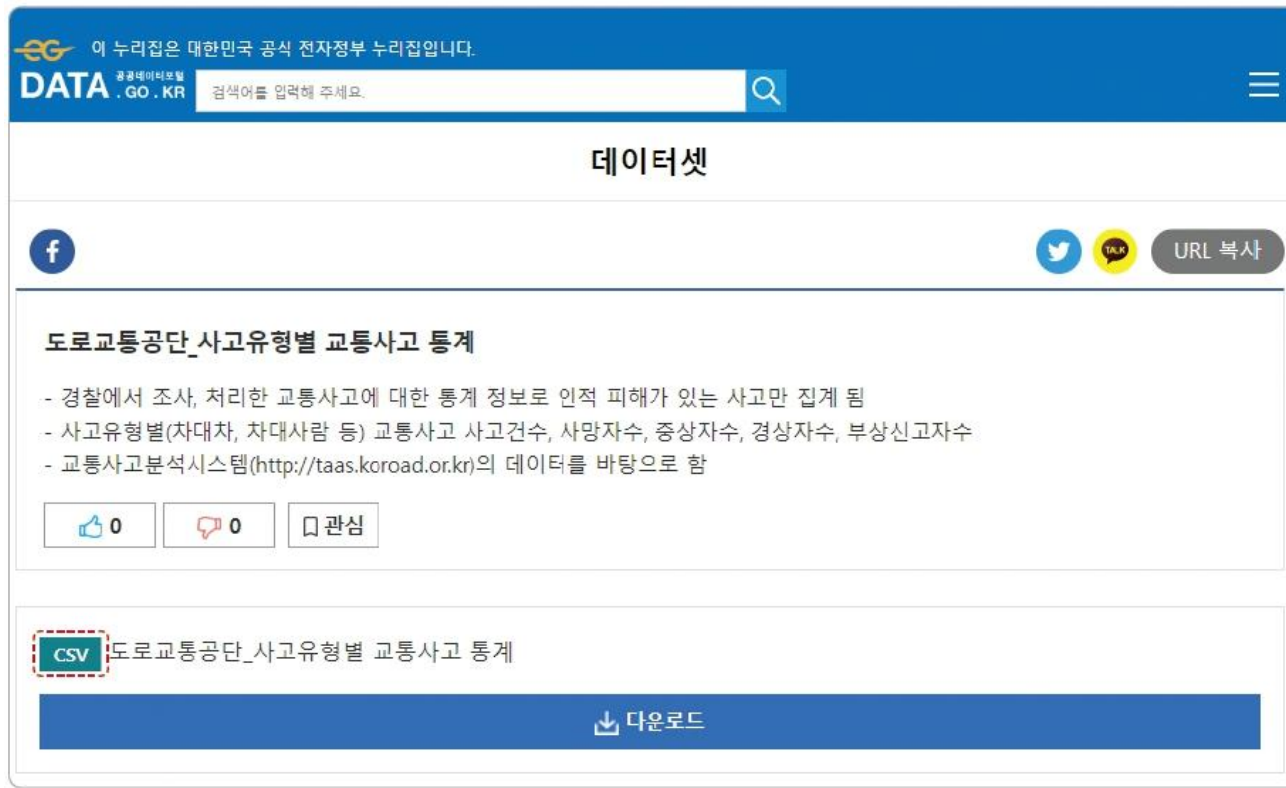
뒤편어쓰기 방지와 예외처리

```
with open('hello.txt', mode='w') as f:
    f.write('Hello world!')
try:
    with open('hello.txt', 'x') as f:
        f.write('Hi~~~')
except FileExistsError as e:
    print(e)
    print('쓰기 실패ㅠㅠ')
else:
    with open('hello.txt', 'r') as f:
        print(f.read())
```

01. CSV 파일

I. CSV 파일 이해하기

- 공공데이터포털(www.data.go.kr) 대부분의 공공데이터 파일은 CSV 형식

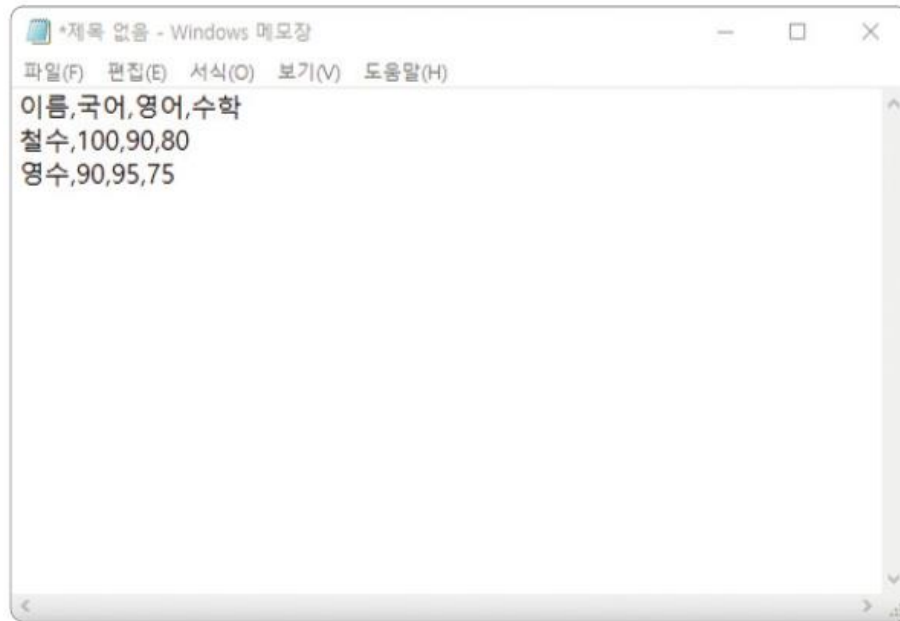


공공데이터포털에서 제공하는 CSV 데이터셋

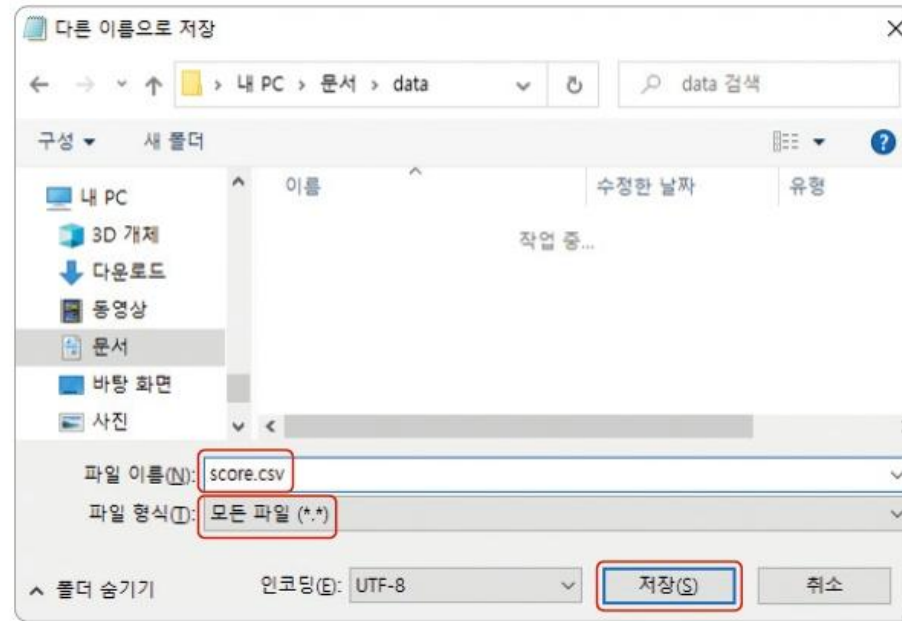
CSV 파일

I. CSV 파일 이해하기

- CSV(Comma-separated Value) 파일은 데이터를 쉼표(,)로 구분하는 텍스트 데이터 형식



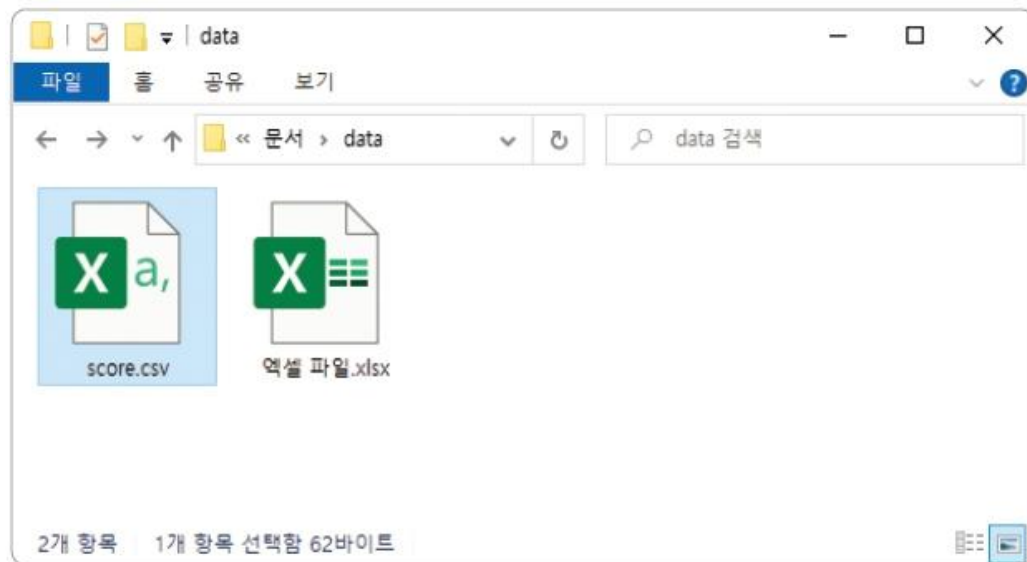
메모장으로 CSV 파일 생성



CSV 파일

I. CSV 파일 이해하기

- PC에 엑셀 프로그램이 설치되어 있다면 'score.csv' 파일이 탐색기에서 엑셀 파일(xlsx) 아이콘과 비슷한 모양으로 나타남



CSV 파일과 엑셀 파일

CSV 파일

I. CSV 파일 이해하기

- 데이터에 한글이 포함된 CSV 파일
- 엑셀 프로그램으로 'score.csv' 파일을 열고 내용을 확인해 보면 그림과 같이 한글이 깨져 보일 것 (인코딩 문제)
- 인코딩(Encoding) : 문자 데이터를 컴퓨터에게 전달하는 방식

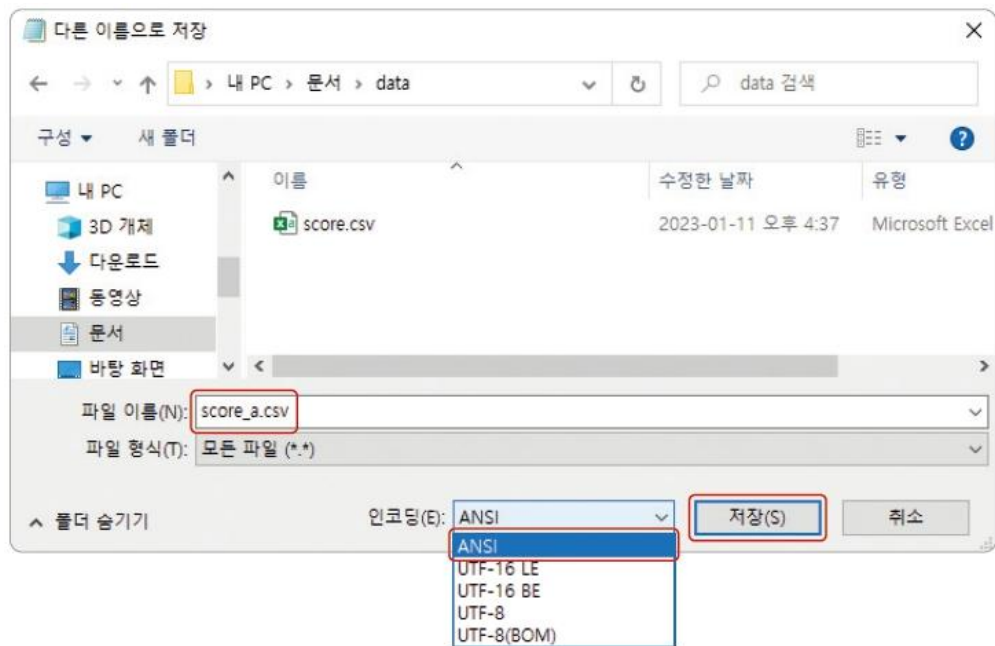
	A	B	C	D
1	?대쫘	援?뵤	?곡뵤	?샹뵤
2	泥쫘뵤	100	90	80
3	?곡뵤	90	95	75

엑셀로 CSV 파일 열기(1)

CSV 파일

I. CSV 파일 이해하기

- 데이터에 한글이 포함된 CSV 파일
- 저장 시 인코딩을 **[ANSI]** 또는 **UTF-8(python)**로 변경하고 파일로 저장



메모장에서 인코딩 변경하기

CSV 파일

■ CSV 파일 이해하기

- 데이터에 한글이 포함된 CSV 파일
- 새로 저장한 'score_a.csv' 파일을 엑셀 프로그램으로 다시 열어보면 한글이 깨지지 않고 제대로 보임

	A	B	C	D
1	이름	국어	영어	수학
2	철수	100	90	80
3	영수	90	95	75

엑셀로 CSV 파일 열기(2)

csv 파일 사용 예

File Edit Format Run Options Window Help

```
1 import csv
2 f= open('G:/score.csv',encoding='UTF-8')
3 data=csv.reader(f)
4
5 header=next(data)
6 for row in data:
7     print(row)
8     avg=sum(map(int,row[1:4]))/len(row[1:4])
9     print('평균 : {:.2f}'.format(avg))
10
11 f.close()
```

score				
파일 편집 보기				
이름,국어,영어,수학				
길동	100	80	70	
몽룡	100	100	100	

실행 결과

```
['길동', '100', '80', '70']
평균 : 83.33
['몽룡', '100', '100', '100']
평균 : 100.00
```

CSV 파일

■ CSV 파일 이해하기

- 데이터에 쉼표가 포함된 CSV 파일
- 데이터에 쉼표(,)가 포함된 경우 유의해야 함

쉼표가 포함된 데이터

품목	가격
딸기,사과	1,000
수박	5,000

CSV 파일

■ 데이터에 쉼표가 포함된 CSV 파일

- 메모장에서 다음과 같이 작성하여 'fruits.csv'로 저장

```
품목,가격  
딸기,사과,1,000  
수박,5,000
```

- 같은 셀에 있어야 할 딸기와 사과가 쉼표 때문에 다른 셀에 나타나며 숫자들도 천 단위로 쉼표가 있어 분리됨

	A	B	C	D
1	품목	가격		
2	딸기	사과	1	0
3	수박	5	0	

쉼표가 포함된 데이터(1)

CSV 파일

I. CSV 파일 이해하기

- 데이터에 쉼표가 포함된 CSV 파일
- 같은 셀에 넣을 데이터에 쉼표가 포함된 경우에는 따옴표로 묶어서 작성

품목, 가격

"딸기, 사과", "1,000"

수박, "5,000"

	A	B
1	품목	가격
2	딸기, 사과	1,000
3	수박	5,000

쉼표가 포함된 데이터(2)

✓ 하나 더 알기: 데이터의 포맷

CSV만큼이나 자주 쓰이는 데이터 파일 형식이 XML과 JSON.

XML과 JSON 파일은 웹 상의 데이터를 나타내는데, 네트워크를 통해 데이터를 주고받을 때 데이터에 마크업(Mark-up).

즉, 문서나 데이터의 구조를 밝힐 수 있어서 데이터를 정제하거나 분석하는 데에 매우 효율적

CSV 파일

II. 파이썬으로 CSV 파일 다루기

- CSV와 리스트
 - 인덱스를 이용하여 'characters.csv' 파일의 데이터 중 두 번째 열의 이름만 출력.

'characters.csv' 파일의 데이터

ID	이름	상징색	취미	특징
001	뽀로로	파랑색	낚시	펭귄
002	에디	주황색	과학실험	사막여우
003	크롱	초록색	눈싸움	공룡
004	루피	분홍색	요리	비버
005	패티	보라색	운동	펭귄

CSV 파일

II. 파이썬으로 CSV 파일 다루기

- CSV와 리스트
 - 인덱스를 이용하여 'characters.csv' 파일의 데이터 중 두 번째 열의 이름만 출력.

[특정 열 출력 코드]

```
import csv  
  
f = open('characters.csv', 'r', encoding = 'UTF8')  
rdr = csv.reader(f)  
  
#두 번째 열을 반복문으로 한 행씩 출력하기  
for line in rdr:  
    print(line[1])
```

[실행결과]

```
이름  
뽀로로  
에디  
크롱  
루피  
패티
```

CSV 파일

II. 파이썬으로 CSV 파일 다루기

- CSV와 리스트
 - 인덱스를 이용하여 'characters.csv' 파일의 데이터 중 두 번째 열의 이름만 출력

[특정 열 출력 코드]

```
f.close( )
```

- 읽기가 끝났어도 변수 f에는 파일을 읽어온 내용이 남아있음
- 다 사용하고 난 후에는 close() 함수를 호출하여 자원을 반환해야 함

II. 파이썬으로 CSV 파일 다루기

[CSV 파일 작성 코드]

```
import csv
f = open('write.csv', 'w', encoding='UTF8', newline= " ")
#CSV 파일 작성할 준비하기
wr = csv.writer(f)
#데이터를 한 행씩 작성하기
wr.writerow( [ 'ID', '이름', '상징색', '취미', '특징' ] )
wr.writerow( [ '001', '뽀로로', '파랑색', '낚시', '펭귄' ] )
wr.writerow( [ '002', '에디', '주황색', '과학실험', '사막여우' ] )
wr.writerow( [ '003', '크롱', '초록색', '눈싸움', '공룡' ] )
wr.writerow( [ '004', '루피', '분홍색', '요리', '비버' ] )
wr.writerow( [ '005', '패티', '보라색', '운동', '펭귄' ] )
f.close()
```

- open() 함수로 빈 파일을 불러오기
- csv.writer() 함수를 호출하여 wr 변수에 CSV 파일을 작성할 준비를 한 다음
writerow() 함수를 호출하여 행별로 데이터를 작성
- f.close()를 호출하여 자원을 반환

CSV 파일

II. 파이썬으로 CSV 파일 다루기

- 데이터 추가하기
 - CSV 파일 마지막 행 아래에 한 행을 추가하여 캐릭터 데이터를 입력

캐릭터 추가

ID	이름	상징색	취미	특징
001	뿌로로	파랑색	낚시	펭귄
002	에디	주황색	과학실험	사막여우
003	크롱	초록색	눈싸움	공룡
004	루피	분홍색	요리	비버
005	패티	보라색	운동	펭귄
006	포비	흰색	춤	북극곰

파이썬으로 CSV 파일에 데이터 추가

[CSV 파일에 데이터 추가]

```
import csv  
  
f = open('write.csv', 'a', encoding='UTF8', newline='')  
  
wr = csv.writer(f)  
  
wr.writerow(['006', '포비', '흰색', '춤', '북극곰'])  
  
#f 변수의 자원 반환하기  
  
f.close()
```

- open() 함수의 두 번째 인자로 a(추가하기)를 사용
- writerow() 함수로 내용을 작성하면 wr의 맨 끝에 추가됨
- 데이터를 추가하는 동작이 끝나면 close() 함수를 호출하여 객체의 자원을 반환

웹페이지 열기

- with 구문 사용
- 웹페이지를 열기 위해서는 urllib이라는 패키지를 사용

코드 9-23 : with 문을 사용한 웹 접속

urllib_with.py

```
import urllib.request

with urllib.request.urlopen('http://python.org/') as response:
    html = response.read()
    print(html)
```

이 문장이 끝나면 자동으로 인터넷 접속을 종료하는 close() 메소드를 호출함

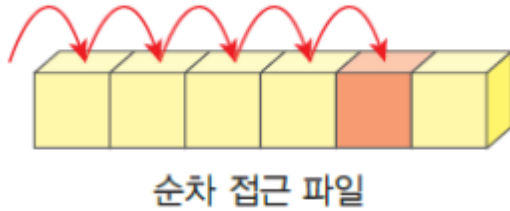
더블클릭 하면 다음과 같은 내용이 보임

실행결과

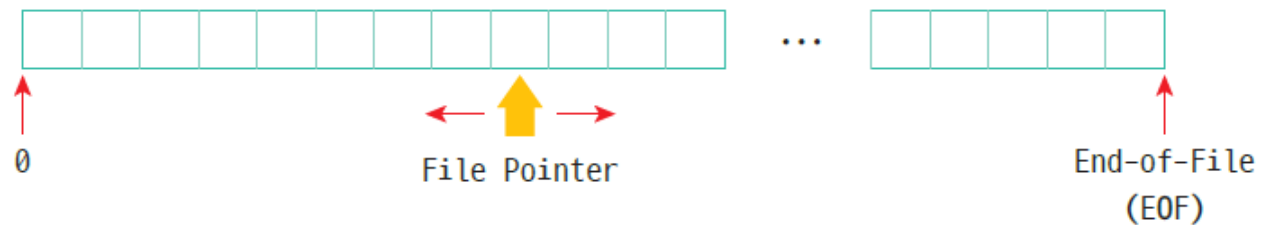
Squeezed text (653 lines).

```
b"<HTML>\r\n<frameset rows='*,0' border=0>\r\n<frame src='http://python.org?'>\r\n
...
</frameset>\r\n</HTML>"
```

순차 접근과 임의 접근



- 파일 포인터 위치 지시자
 - 읽기와 쓰기 동작이 현재 어느 위치에서 이루어지는 지를 나타냄
 - 파일에서 읽거나 쓰기가 수행되면 파일 포인터가 갱신



임의 접근

- seek() 메소드

- seek(offset, whence)

- whence : 기준 (0-파일의 처음, 1-현재 위치, 2-파일의 끝)

- offset : 이동 바이트 수

- 예)

- 1. f.seek(n, 0) # 파일의 n번째 바이트로 이동

- 2. f.seek(n, 1) # 현재 위치에서 n번 바이트로 이동

- (n양수 일 경우 뒤쪽으로 이동, n이 음수일 경우 앞쪽으로 이동)

- 3. f.seek(n, 2) # 파일의 끝에서 n번째 바이트로 이동

- ❖ Text 모드 파일은 기준위치를 처음(0)만 지원

임의 접근 예

seek예.py - G:\파이썬3\강의자료2024_1학기\seek예.py (3.12.1)

File Edit Format Run Options Window Help

```
1 infile = open("alphabet.txt", "r+")
2 str = infile.read(10);
3 print("읽은 문자열 : ", str)
4 position = infile.tell();
5 print("현재 위치: ", position)
6
7 position = infile.seek(20)
8 str = infile.read(5);
9 print("읽은 문자열 : ", str)
10 infile.close()
```

alphabet

파일 편집 보기

abcdefghijklmnopqrstuvwxyz

IDLE Shell 3.12.1

File Edit Shell Debug Options Window Help

```
Python 3.12.1 (tags/v3.12.1:23
AMD64)] on win32
Type "help", "copyright", "cre
>>>
= RESTART: G:\W파이썬3\강의 자료
읽은 문자열 : abcdefghij
현재 위치: 10
읽은 문자열 : uvwxy
```

객체 파일 쓰기(바이너리)

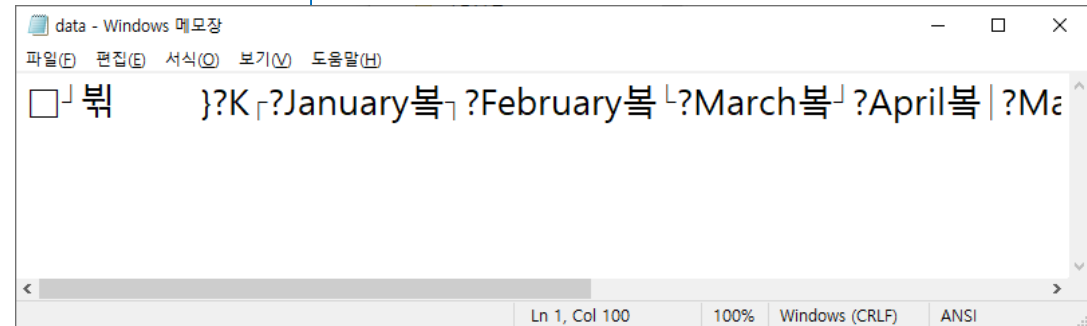
- 표준 모듈 `pickle`을 사용, mode='wb'로 지정
- 객체를 바이너리 형태로 파일에 출력할 때 함수 `dump(자료, 파일)` 사용
- 바이너리 파일의 특징은 메모장, 노트패드와 같은 텍스트 편집기로는 내용을 볼 수 없음

```
import pickle as pk

month = {1: 'January', 2: 'February', 3: 'March', 4: 'April'}
month[5] = 'May'
month[6] = 'June'
lst = ['pascal', 'python', 'java']

#바이너리 파일 쓰기
with open('month_data.bin', mode='wb') as f:
    pk.dump(month, f)
    pk.dump(lst, f)

print(' 바이너리 파일 쓰기 완료! '.center(30, '*'))
```



객체 읽기(바이너리)

- load(파일객체) 메소드 사용

```
import pickle as pk

#바이너리 파일 읽기
try:
    with open('month_data.bin', mode='rb') as f:
        dmon = pk.load(f)
        pl = pk.load(f)
except FileNotFoundError as e:
    print(e)
    print(' 파일 읽기 실패! '.center(30, '*'))
else:
    print(dmon)
    print(pl)
    print(' 바이너리 파일 읽기 완료! '.center(30, '*'))
finally:
    print(' 프로그램 종료! '.center(30, '*'))
```



Questions?